

Arrays and Functions

Discussing Arrays and Functions through Code Snippets and Outcome-Based Education (OBE) Framework

Subject: Programming for Problem Solving (ESCS201) • **Program:** B.Tech Computer Science & Engineering • **Semester:** 2 • **Department:** Computer Science & Engineering
• **Faculty:** Dr. A. Sharma • **AY:** 2026-2027 • **Date:** August 3, 2026

Outcome-Based Education (OBE) Framework

1. Course Outcomes (COs)

Course Outcomes for Module 2: Arrays & Memory Management

- CO1** Understand foundational concepts of linear data structures and contiguous memory allocation in C/C++. **L2 Understand PO1**
- CO2** Apply pointer arithmetic and dynamic memory allocation routines (malloc/free) to manage memory efficiently. **L3 Apply PO2**
- CO3** Analyze and compare time and space complexities of array manipulation algorithms (Search & Insertion). **L4 Analyze PO3**
- CO4** Design and implement modular functions handling single and multi-dimensional array operations. **L6 Create PO4**

2. CO-PO Articulation Matrix

CO-PO Articulation Matrix

CO / PO	PO1	PO2	PO3	PO4	PO5	PO6
CO1	3	2	1	-	-	-
CO2	3	3	2	1	-	-
CO3	2	3	3	2	1	-
CO4	2	2	3	3	2	1

Legend: 3 - High / Substantial Correlation, 2 - Medium / Moderate Correlation, 1 - Low / Slight Correlation

3. Cognitive Domain Reference

Bloom's Taxonomy Cognitive Levels

L1 Remember

Verbs: Define, List, State, Recall, Identify

L2 Understand

Verbs: Explain, Describe, Classify, Discuss, Summarize

L3 Apply

Verbs: Solve, Calculate, Demonstrate, Implement, Use

L4 Analyze

Verbs: Differentiate, Compare, Contrast, Examine, Deconstruct

L5 Evaluate

Verbs: Assess, Judge, Critique, Justify, Validate

L6 Create

Verbs: Design, Construct, Formulate, Develop, Synthesize

1. Introduction to Arrays C01 L2 Understand

An **array** is a contiguous collection of elements of the same data type stored sequentially in memory. Arrays allow constant-time $O(1)$ index-based random access.

Definition (Array):

An array is a fixed-size, homogeneous data structure that stores elements in contiguous memory locations.

□ Best Practice

Always ensure that array access stays within bounds `[0, size - 1]` to avoid undefined behavior or segmentation faults in languages like C/C++.

★ **Key Takeaway:** Array indices start at `0` in zero-indexed languages. The address of element `arr[i]` is computed as:

$$\text{Address}(\text{arr}[i]) = \text{Base Address} + i \times \text{sizeof}(\text{element})$$

Code Example: Dynamic vs Static Allocation C02 L3 Apply

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    // Dynamic memory allocation for array
```

```
int *arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) return 1;

for (int i = 0; i < n; i++) {
    arr[i] = (i + 1) * 10;
    printf("arr[%d] = %d\n", i, arr[i]);
}

free(arr);
return 0;
}
```

2. Time Complexity & Theoretical Foundations C03

L4 Analyze

Theorem 1 (Time Complexity of Linear Search):

Given an un-ordered array of size N , searching for a target element using Linear Search requires $O(N)$ comparisons in the worst case.

Proof: In the worst-case scenario, the target element is located at the last index $N - 1$ or is not present in the array at all. The algorithm must inspect every element from index 0 up to $N - 1$, performing exactly N comparisons. Thus, the time complexity is $O(N)$. □

Corollary (Sorted Array Search):

If the array is sorted, Binary Search reduces the search time complexity to $O(\log N)$.

Example 1 (Array Access Efficiency):

Consider a 1D array `int arr[1000]`. Accessing `arr[42]` takes $O(1)$ time because the memory address is calculated directly via pointer arithmetic.

3. Summary Comparison

Operation	Static Array	Dynamic Array	Notes
Index Access	$O(1)$	$O(1)$	Direct pointer offset calculation
Insertion at End	$O(1)^*$	$O(1)$ amortized	Requires space check for static
Search (Unsorted)	$O(N)$	$O(N)$	Requires full linear scan
Search (Sorted)	$O(\log N)$	$O(\log N)$	Binary search algorithm

⚠ Memory Allocation Note

Un-freed dynamic allocations result in memory leaks. Always pair `malloc()` / `calloc()` with `free()`.

📌 **TODO: Add section on multi-dimensional array mapping functions in column-major vs row-major order.**

4. OBE Practice Questions & Assessments

Question 1

CO1

L2

Understand

PO1

5 pts

What is the primary advantage of arrays over linked lists regarding memory locality?

Answer:

Arrays store elements in contiguous memory locations, which maximizes CPU cache hit ratios due to spatial locality of reference.

Question 2

CO2

L3

Apply

PO2

5 pts

Explain why passing a large array to a function in C by value is inefficient.

Answer:

Passing by value copies the entire array contents onto the call stack, causing $O(N)$ memory copy overhead. Passing by reference (pointer) passes only the 8-byte memory address.

Question 3

CO4

L6

Create

PO4

10 pts

Write a modular C function `reverseArray(int *arr, int size)` to reverse an array in-place without using auxiliary memory, and trace its performance on an array of size $N = 6$.

Answer:

```
void reverseArray(int *arr, int size) {
    int start = 0, end = size - 1;
    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
    }
}
```

```

        start++; end--;
    }
}

```

Evaluation Rubric / Marking Scheme:

- **Correct Function Signature & In-Place Swap Logic:** 5 Marks
- **Pointer / Index Boundary Checks:** 3 Marks
- **Execution Trace on N=6 Example:** 2 Marks

Quick Self-Check Q&A

Q1: What is the time complexity of inserting an element at the beginning of an array?

A: $O(N)$ because all existing elements must be shifted one index to the right.

Q2: What does `sizeof(arr) / sizeof(arr[0])` evaluate to for a statically allocated array?

A: It evaluates to the total number of elements in the static array.

5. OBE Outcome Coverage Summary

OBE Assessment Outcome Summary

CO Code	Course Outcome Description	Bloom's Level	Questions	Weightage
CO1	Understand contiguous memory & data structure basics	L2 Understand	1	5 pts
CO2	Apply dynamic memory allocation & pointer routines	L3 Apply	1	5 pts
CO3	Analyze searching algorithms and runtime complexity	L4 Analyze	1	0 pts
CO4	Design & implement in-place modular array routines	L6 Create	1	10 pts